

Claude Code

Полное руководство по AI-ассистенту
для разработчиков

```
class Model Gueans {  
  def __adinaace:objects {  
    constd.banfae = "achanoa-kids";  
  
    function noteRivaluel() {  
      Boot:print("Chaude nyon post stangs");  
      return hows();  
    }  
  
    export 3rgoBeerts rcehooonex {  
      return as.Second();  
    }  
}
```

Ранас Мукминов
Claude Code: Полное
руководство по AI-
ассистенту для разработчиков

*http://www.litres.ru/pages/biblio_book/?art=73415023
SelfPub; 2026*

Аннотация

включает описание книги и перечень ключевых тем: написание кода, рефакторинг, отладка, тесты, MCP, DevOps, безопасность, реальные проекты.

Содержание

Предисловие	4
Часть I. Введение и основы	9
Глава 1. Что такое Claude Code и зачем он нужен	9
Глава 2. Установка и настройка окружения	34
Глава 3. Первые шаги с Claude Code	67
Глава 4. Архитектура и принципы работы	105
Конец ознакомительного фрагмента.	115

Ранас Мукминов

Claude Code: Полное

руководство по

AI-ассистенту для

разработчиков

Предисловие

О чём эта книга

Уважаемый читатель, перед вами – наиболее полное русскоязычное руководство по Claude Code, революционному AI-ассистенту для разработчиков, созданному компанией Anthropic. Эта книга родилась из простого наблюдения: мир программирования переживает фундаментальный сдвиг, сопоставимый по масштабу с появлением объектно-ориентированного программирования, открытого программного обеспечения или облачных вычислений. Искусственный интеллект больше не является экзотической технологией из на-

учно-фантастических фильмов – он превратился в повседневный инструмент, способный кардинально изменить то, как мы пишем, отлаживаем и поддерживаем программный код.

Claude Code занимает особое место среди AI-инструментов для разработки. В отличие от простых автодополнителей кода, Claude Code представляет собой полноценного агентного ассистента, способного понимать контекст вашего проекта, работать с файловой системой, запускать команды в терминале, анализировать ошибки и самостоятельно планировать сложные многоэтапные задачи. Это не просто «умная подсказка» – это настоящий напарник по программированию, который может взять на себя существенную часть рутинной работы.

Для кого эта книга

Книга адресована широкому кругу разработчиков:

– **Начинающим программистам**, которые хотят ускорить свой рост, используя AI как наставника и помощника. Claude Code может объяснить сложные концепции, предложить лучшие практики и помочь быстро прототипировать идеи.

– **Опытным разработчикам**, стремящимся повысить свою продуктивность. Если вы тратите часы на рутинный рефакторинг, написание шаблонного кода или отладку типичных ошибок, Claude Code поможет автоматизировать эти задачи.

– **Тимлидам и архитекторам**, заинтересованным во внедрении AI-инструментов в рабочие процессы команды. Книга содержит практические рекомендации по настройке, стандартизации и масштабированию использования Claude Code.

– **Менеджерам и принимающим решения**, которым важно понимать возможности и ограничения AI-ассистентов для оценки ROI и планирования трансформации процессов разработки.

Как читать эту книгу

Книга организована в пять логических частей, которые можно читать последовательно или использовать как справочник:

Часть I закладывает фундамент: от установки и первых

шагов до глубокого понимания архитектуры Claude Code и искусства эффективного общения с AI.

Часть II погружает в практику: написание кода, рефакторинг, отладка, тестирование, работа с базами данных, DevOps и Git – все основные аспекты ежедневной работы разработчика.

Часть III раскрывает продвинутые техники: MCP (Model Context Protocol), автоматизацию, безопасность, мультитяжечную разработку, архитектурное проектирование и работу с API.

Часть IV представляет шесть полноценных проектов, которые вы можете воспроизвести от начала до конца: SaaS-платформа, мобильное приложение, CLI-утилита, Telegram-бот, Data Pipeline и микросервисная система.

Часть V выходит за рамки технологии: командная работа, сравнение инструментов, этические вопросы, оптимизация продуктивности и взгляд в будущее.

Каждая глава содержит практические примеры, фрагменты кода, советы и предупреждения. Я рекомендую держать под рукой терминал с установленным Claude Code и пробовать всё на практике.

Благодарности

Эта книга не появилась бы без открытости и прозрачности команды Anthropic, создавшей Claude и Claude Code. Отдельная благодарность – сообществу разработчиков, которые активно делятся опытом использования AI-ассистентов и формируют лучшие практики.

Надеюсь, после прочтения этой книги вы не просто научитесь использовать Claude Code, но и переосмыслите свой подход к разработке программного обеспечения. Добро пожаловать в эру AI-ассистированного программирования!

Автор, февраль 2026 года

Часть I. Введение и основы

Глава 1. Что такое Claude Code и зачем он нужен

1.1. Краткая история Anthropic и линейки Claude

Чтобы понять место Claude Code в современном ландшафте разработки, необходимо начать с истории компании, которая за ним стоит. Anthropic была основана в 2021 году группой исследователей, ранее работавших в OpenAI. Среди основателей – Дарио Амодей, занимавший пост вице-президента по исследованиям в OpenAI, и его сестра Даниэла Амодей, отвечавшая за операционное управление. Вместе с ними компанию покинули несколько ключевых исследователей, объединённых общей идеей: создать AI-системы, которые будут не только мощными, но и безопасными, предсказуемыми и управляемыми.

С момента основания Anthropic выбрала иной путь развития, нежели большинство конкурентов. Вместо гонки за

размером моделей и бенчмарками компания сфокусировалась на том, что её основатели называют «конституционным AI» (Constitutional AI, или CAI). Суть подхода заключается в том, что модель обучается следовать набору принципов, сформулированных на естественном языке, – своего рода «конституции», определяющей границы допустимого поведения. Это позволяет создавать системы, которые не просто генерируют текст, но делают это ответственно, с пониманием контекста и последствий.

Первая публичная модель Anthropic – Claude – появилась в марте 2023 года. Она сразу привлекла внимание сообщества качеством генерации текста, способностью к анализу и рассуждению, а также заметно более аккуратным обращением с потенциально вредоносными запросами. В июле 2023 года вышла Claude 2, продемонстрировавшая значительный скачок в качестве кода, математических способностях и работе с длинными документами – контекстное окно расширилось до ста тысяч токенов.

Модель Claude 3 стала переломным моментом. Выпущенная в марте 2024 года в трёх вариантах – Haiku, Sonnet и Opus, – она показала результаты, сопоставимые и зачастую превосходящие GPT-4 от OpenAI на множестве бенчмарков. Claude 3 Opus заняла лидирующие позиции в задачах, требующих глубокого рассуждения и анализа, а более компактный

Sonnet нашёл оптимальный баланс между скоростью, стоимостью и качеством.

Линейка продолжила развитие: Claude 3.5 Sonnet, выпущенная летом 2024 года, установила новые стандарты в программировании, превзойдя более дорогие модели-конкуренты на задачах генерации и анализа кода. Именно с этой модели началась эпоха Claude как серьёзного инструмента для профессиональной разработки. Но настоящая революция произошла с выпуском Claude Code.

Claude Code – это не просто API для генерации текста и кода. Это агентный инструмент командной строки, способный работать непосредственно в вашей среде разработки. Он читает файлы, пишет код, запускает тесты, анализирует ошибки, работает с системой контроля версий, управляет зависимостями и взаимодействует с внешними сервисами. Claude Code превращает мощную языковую модель в полноценного участника процесса разработки, который «живёт» в вашем терминале и имеет доступ к вашему проекту.

1.2. Место Claude Code среди AI-инструментов для разработки

Рынок AI-инструментов для разработки к 2026 году стал

чрезвычайно насыщенным. Чтобы понять уникальную позицию Claude Code, необходимо рассмотреть основные категории существующих решений.

Автодополнители кода – первая и наиболее массовая категория. GitHub Copilot, появившийся в 2021 году, стал пионером этого направления. Принцип прост: вы пишете код, а AI предлагает продолжение – от одной строки до целого блока. К этой же категории относятся Codeium (бесплатная альтернатива), TabNine (использующий как облачные, так и локальные модели) и Amazon CodeWhisperer. Автодополнители интегрируются непосредственно в редактор и работают на уровне «следующей строки». Они хороши для ускорения набора типового кода, но их возможности ограничены: они не понимают архитектуру проекта в целом, не могут выполнить сложную многошаговую задачу и не имеют доступа к файловой системе или терминалу.

AI-ориентированные IDE – вторая категория, представленная прежде всего Cursor. Cursor взял код Visual Studio Code и перестроил его вокруг AI: модель имеет доступ ко всей кодовой базе, может предлагать изменения в нескольких файлах одновременно, понимает контекст проекта и поддерживает диалог в чате. Windsurf (панель Codeium IDE) и Void предлагают аналогичный подход. Эти инструменты мощнее простых автодополнителей, но привязаны к

конкретному редактору и его парадигме взаимодействия.

Чат-боты для кода – третья категория. ChatGPT, Gemini, Perplexity и другие универсальные AI-системы могут генерировать код, объяснять концепции и помогать с отладкой. Однако они работают в изоляции от вашего проекта: вы вручную копируете код в чат и обратно, AI не видит вашу файловую систему и не может запустить тест или проверить результат своей работы.

Агентные инструменты – четвёртая и новейшая категория, к которой относится Claude Code. Отличие агентных инструментов от всех предыдущих категорий фундаментально: они не просто генерируют код, а действуют. Агентный AI-ассистент может:

- Самостоятельно исследовать кодовую базу, открывая и анализируя нужные файлы
- Планировать последовательность шагов для решения сложной задачи
- Создавать и модифицировать файлы непосредственно на диске
- Запускать команды в терминале и анализировать их вывод
- Работать с Git: создавать ветки, коммитить изменения, разрешать конфликты

- Запускать тесты и исправлять ошибки на основе результатов
- Взаимодействовать с внешними сервисами через MCP (Model Context Protocol)
- Открывать браузер для проверки веб-приложений

Claude Code работает из командной строки, что делает его универсальным: ему не нужен конкретный редактор, он функционирует на любой системе с Node.js и терминалом. Вы можете использовать его с Vim, Emacs, VS Code, JetBrains или любым другим редактором. Вы даже можете использовать его на удалённом сервере через SSH.

Принципиальное отличие Claude Code от конкурентов можно проиллюстрировать простым примером. Допустим, вам нужно добавить аутентификацию в веб-приложение. Автодополнитель поможет сгенерировать отдельные функции. Чат-бот покажет примерный код, который вы скопируете вручную. AI-ориентированный IDE может внести изменения в несколько файлов. Но Claude Code проанализирует вашу существующую архитектуру, определит нужные файлы, создаст модели, маршруты, middleware, напишет тесты, обновит зависимости, запустит тесты для проверки, исправит обнаруженные ошибки и предложит закоммитить результат – всё это в рамках одного диалога, без переключения контекста.

1.3. Подробное сравнение с основными конкурентами

Давайте рассмотрим каждого конкурента более детально, чтобы понять сильные и слабые стороны Claude Code в контексте каждого из них.

GitHub Copilot

GitHub Copilot – безусловно, самый массовый AI-инструмент для разработки. По состоянию на начало 2026 года он насчитывает миллионы активных пользователей. Copilot интегрируется в популярные IDE и предлагает автодополнение кода на основе моделей GPT от OpenAI. С выходом Copilot Chat и Copilot Workspace Microsoft расширила функциональность, добавив диалоговый интерфейс и возможность работы с несколькими файлами.

Преимущества Copilot:

- Глубокая интеграция с экосистемой GitHub
- Огромная база пользователей и отработанный продукт

- Поддержка множества языков и редакторов
- Относительно низкая стоимость подписки
- Copilot Workspace для планирования изменений

Преимущества Claude Code по сравнению с Copilot:

- Полноценный агентный режим с доступом к файловой системе и терминалу
- Значительно более длинное контекстное окно (200К+ токенов vs ~8К у Copilot для автодополнения)
- Способность к сложному рассуждению и планированию
- Не привязан к конкретному редактору или экосистеме
- MCP для интеграции с любыми внешними сервисами
- Лучшее понимание архитектуры и паттернов проектирования
- Возможность работы в полностью автономном режиме

Cursor IDE

Cursor – это, пожалуй, наиболее близкий конкурент Claude Code по философии подхода. Cursor перестроил VS Code вокруг AI, предоставив модели доступ ко всей кодовой базе и возможность вносить изменения в несколько файлов одновременно. Cursor поддерживает несколько моделей, включая, иронично, Claude Sonnet от Anthropic.

Преимущества Cursor:

- Полноценная IDE с визуальным интерфейсом
- Встроенный diff-просмотрщик для изменений AI
- Возможность выбора между разными моделями
- Удобная работа с множественными файлами
- Tab-completion и встроенный чат

Преимущества Claude Code по сравнению с Cursor:

- Не требует смены редактора – работает из терминала
- Полноценное выполнение команд и анализ результатов
- MCP-экосистема для расширения возможностей
- Более глубокое понимание контекста проекта
- Лучшая поддержка автоматизации и скриптинга
- Работа на удалённых серверах через SSH
- Прозрачная модель ценообразования (API-вызовы)

Amazon CodeWhisperer (Q Developer)

Amazon переименовала CodeWhisperer в Q Developer, расширив функциональность за пределы простого автодополнения. Q Developer предлагает анализ безопасности, генерацию тестов и интеграцию с сервисами AWS.

Преимущества Q Developer:

- Бесплатный для индивидуального использования
- Глубокая интеграция с AWS
- Встроенный анализ безопасности кода
- Поддержка многих языков

Преимущества Claude Code:

- Значительно более мощная базовая модель
- Агентные возможности
- Универсальность: не привязан к облачному провайдеру
- MCP для интеграций
- Лучшее качество генерации кода на большинстве язы-

ков

Google Gemini Code Assist

Google предлагает Gemini Code Assist как инструмент для разработки, интегрированный с IDE и облачной платформой Google Cloud. Используя модели семейства Gemini, инструмент обеспечивает автодополнение, чат и анализ кода.

Преимущества Gemini Code Assist:

- Огромное контекстное окно (до 1М+ токенов)
- Интеграция с Google Cloud
- Бесплатный уровень использования
- Мультимодальность (понимание изображений и скриншотов)

Преимущества Claude Code:

- Агентный режим с реальным выполнением действий
- Лучшее качество кода в большинстве сценариев
- MCP-экосистема
- Не привязан к облачному провайдеру
- Более зрелая архитектура безопасности

1.4. Философия Claude Code: AI как напарник

За Claude Code стоит определённая философия разработки, которую важно понять для эффективного использования инструмента. Anthropic позиционирует Claude Code не как замену разработчика, а как «напарника» – равноправного участника процесса разработки, который берёт на себя выполнение задач, но оставляет контроль и принятие решений человеку.

Это отличается от двух крайних подходов, которые можно наблюдать на рынке:

Подход «умное автодополнение» – AI как прокачанная версия IntelliSense. Он просто предлагает следующую строку кода, не вмешиваясь в процесс. Минимальный риск, но и минимальная трансформация рабочего процесса.

Подход «полная автономия» – AI как самостоятельный разработчик, который получает задачу и возвращает готовый результат. Максимальная автоматизация, но и максимальные риски: потеря контроля, непредсказуемые решения, сложность отладки.

Claude Code занимает промежуточную позицию. Он достаточно автономен, чтобы самостоятельно планировать и выполнять сложные задачи, но при этом:

- **Запрашивает подтверждение** перед потенциально опасными операциями (удаление файлов, установка зависимостей, выполнение неизвестных команд)
- **Объясняет свои действия** – каждый шаг сопровождается описанием того, что делается и зачем
- **Сохраняет прозрачность** – вы видите все инструменты, которые вызывает AI, и все изменения, которые он вносит

– **Допускает итерации** – вы можете скорректировать направление в любой момент

– **Уважает границы** – система разрешений позволяет точно определить, что AI может и не может делать

Эта философия выражена в концепции «HITL» (Human-In-The-Loop) – человек остаётся в цикле принятия решений. Вы не просто «запускаете и забываете» – вы руководите процессом, делегируя рутину и концентрируясь на стратегических решениях.

Практические следствия этой философии:

– **Claude Code лучше работает в диалоге.** Вместо того чтобы пытаться описать задачу одним огромным промптом, эффективнее вести итеративный диалог: поставить задачу, посмотреть промежуточный результат, скорректировать направление.

– **Claude Code уважает ваш стиль.** Он анализирует существующий код и старается следовать вашим соглашениям по именованию, форматированию и архитектурным паттернам.

– **Claude Code признаёт ошибки.** Если что-то пошло не так, он не будет настаивать на неверном решении – он

переоценит ситуацию и предложит альтернативу.

– **Claude Code учится из контекста.** Файл CLAUDE.md в корне проекта позволяет задать AI постоянные инструкции: стилевые соглашения, архитектурные решения, бизнес-контекст.

1.5. Сценарии использования: от обучения до продакшена

Claude Code применяется в удивительно широком спектре ситуаций. Вот основные сценарии, каждый из которых подробно раскрывается в последующих главах:

Быстрое прототипирование

Вы придумали стартап и хотите за вечер собрать MVP? Claude Code – идеальный инструмент. Вы описываете идею, и ассистент генерирует структуру проекта, настраивает зависимости, пишет базовый функционал. За несколько часов работы можно получить рабочее веб-приложение, которое обычно потребовало бы дни или недели разработки.

Пример: «Создай веб-приложение для управления задачами с аутентификацией через Google OAuth, drag-and-drop досками в стиле Kanban, и возможностью назначать задачи участникам команды. Используй Next.js, Prisma и PostgreSQL».

Claude Code не просто сгенерирует код – он создаст файловую структуру, напишет схему базы данных, реализует API, фронтенд, настроит аутентификацию и напишет базовые тесты. Разумеется, результат потребует доработки, но как отправная точка он бесценен.

Рефакторинг и модернизация

Если у вас есть legacy-проект на устаревшем стеке, Claude Code поможет с миграцией. Он может:

- Перевести JavaScript-код на TypeScript, добавив типы
- Мигрировать с Class Components на React Hooks
- Обновить Python 2 код до Python 3
- Заменить устаревшие API на современные альтернативы
- Реорганизовать файловую структуру проекта
- Извлечь повторяющийся код в переиспользуемые модули

При этом Claude Code анализирует весь проект целиком, а не отдельные файлы, что позволяет ему делать согласованные изменения и не ломать зависимости.

Отладка сложных проблем

Когда вы столкнулись с багом, который не можете найти уже несколько часов, Claude Code может стать свежим взглядом. Он анализирует stack traces, лог-файлы, исходный код и часто находит причину проблемы за считанные минуты. Особенно ценно это для:

- Race conditions в многопоточном коде
- Memory leaks в долго работающих процессах
- Сложных зависимостей и конфликтов версий
- Проблем конфигурации в DevOps-стеке
- Ошибок в SQL-запросах и схемах данных

Написание тестов

Генерация тестов – одна из наиболее впечатляющих возможностей Claude Code. Разработчики часто откладывают

написание тестов как скучную, рутинную работу. Claude Code может:

- Написать unit-тесты для существующего кода
- Создать интеграционные тесты для API
- Сгенерировать E2E-тесты для веб-приложений
- Добавить тесты для граничных случаев, которые вы могли упустить
- Настроить CI-пайплайн для автоматического запуска тестов

Документирование

Claude Code отлично справляется с задачами документирования:

- Генерация JSDoc/Docstring/KDoc комментариев для функций
- Создание README.md для проектов
- Написание API-документации
- Создание руководств для разработчиков
- Генерация CHANGELOG на основе коммитов
- Документирование архитектурных решений (ADR)

DevOps и инфраструктура

Настройка инфраструктуры – область, где Claude Code проявляет себя особенно ярко:

- Написание Dockerfile и docker-compose.yml
- Создание Kubernetes-манифестов и Helm-чартов
- Настройка CI/CD пайплайнов (GitHub Actions, GitLab CI, Jenkins)
- Terraform-конфигурации для облачных ресурсов
- Настройка веб-серверов (Nginx, Apache, Caddy)
- Конфигурация мониторинга (Prometheus, Grafana)

Обучение и менторство

Для начинающих разработчиков Claude Code служит терпеливым наставником:

- Объясняет сложные концепции на понятном языке
- Показывает лучшие практики на примерах
- Разбирает паттерны проектирования
- Помогает понять чужой код
- Предлагает альтернативные подходы с объяснением

1.6. Ограничения и реалистичные ожидания

Было бы нечестно и непрофессионально представлять Claude Code как волшебную палочку, решающую все проблемы. У инструмента есть объективные ограничения, которые важно понимать с самого начала:

Ограничение контекстного окна

Несмотря на впечатляющие 200 000 токенов контекстного окна, для крупных проектов этого может быть недостаточно. Проект с тысячами файлов и сотнями тысяч строк кода физически не помещается в контекст целиком. Claude Code решает эту задачу с помощью умного поиска и выборочного чтения файлов, но иногда может не увидеть важные зависимости или связи между далёкими частями кодовой базы.

Рекомендация: для больших проектов разбивайте задачи на локальные, затрагивающие ограниченное количество файлов. Используйте CLAUDE.md для предоставления высокоуровневого контекста об архитектуре.

Галлюцинации

Все языковые модели подвержены так называемым галлюцинациям – ситуациям, когда AI генерирует правдоподобный, но неверный ответ. Claude Code может:

- Предложить несуществующую функцию стандартной библиотеки
- Неверно описать поведение API
- Сгенерировать код с тонкими логическими ошибками
- Уверенно утверждать неверный факт

Рекомендация: всегда проверяйте критически важный код. Запускайте тесты, проводите code review, не принимайте сгенерированный код на веру. Claude Code – это черновик, который нужно проверять, а не готовый продукт.

Стоимость

Claude Code использует API Anthropic, который тарифицируется по количеству токенов. Интенсивная разработка может обойтись в существенные суммы. Особенно затратны

операции с большим контекстом – чем больше файлов загружается, тем дороже каждый запрос.

Рекомендация: используйте команду `/compact` для сжатия контекста, избегайте загрузки ненужных файлов, планируйте задачи заранее, чтобы минимизировать количество итераций.

Актуальность знаний

Модель Claude обучена на данных до определённой точки отсечения. Это означает, что она может не знать о самых новых библиотеках, недавних изменениях в API или свежих уязвимостях безопасности. Claude Code частично компенсирует это, читая актуальный код и документацию из вашего проекта, но не имеет доступа к интернету для проверки информации (если не настроен MCP-сервер для этого).

Рекомендация: для работы с новейшими технологиями предоставляйте Claude Code актуальную документацию – добавляйте её в проект или передавайте через промпт.

Специфические домены

Claude Code лучше всего работает с мейнстрим-технологиями: Python, JavaScript/TypeScript, Go, Java, Rust, C/C++. Для нишевых языков (COBOL, Fortran, Erlang, Haskell) и специализированных фреймворков качество генерации может быть ниже.

Рекомендация: для редких технологий предоставляйте больше контекста – примеры кода, документацию, конвенции именования.

1.7. Что вы получите от этой книги

Прочитав эту книгу и применив описанные в ней техники на практике, вы:

- **Освоите Claude Code от установки до продвинутых сценариев.** Вы будете уверенно использовать все возможности инструмента, включая MCP, hooks, автоматизацию и командную работу.

- **Научитесь эффективно общаться с AI.** Составле-

ние промптов – это навык, и эта книга научит вас формулировать задачи так, чтобы получать максимально качественный результат с минимальным количеством итераций.

– **Построите шесть полноценных проектов.** Четвёртая часть книги – это практикум, где вы шаг за шагом создадите SaaS-платформу, мобильное приложение, CLI-утилиту, Telegram-бота, Data Pipeline и микросервисную систему.

– **Поймёте архитектурные принципы.** Claude Code – отличный инструмент для изучения архитектуры программного обеспечения, и книга использует его именно в этом контексте: каждый проект иллюстрирует определённые архитектурные паттерны.

– **Сможете внедрить Claude Code в команде.** Главы о командной работе, безопасности и стандартизации дадут вам необходимые инструменты для этого.

– **Получите стратегическое видение.** Последние главы помогут вам оценить, как AI-ассистенты изменят профессию разработчика в ближайшие годы, и подготовиться к этим изменениям.

1.8. Структура глав и условные обозначения

Каждая глава этой книги следует единой структуре:

- **Введение** – постановка задачи и мотивация
- **Теоретическая часть** – концепции и принципы
- **Практическая часть** – пошаговые инструкции с примерами кода
- **Советы и предупреждения** – типичные ошибки и лучшие практики
- **Итоги** – краткое резюме ключевых моментов

В тексте используются следующие условные обозначения:

Блоки кода содержат фрагменты, которые вы можете вводить в терминал или редактор. Комментарии внутри блоков кода объясняют ключевые моменты.

Промпты – примеры запросов к Claude Code – выделены курсивом и предваряются символом «>».

Совет – полезная рекомендация, которая поможет работать эффективнее.

Предупреждение – потенциальная ловушка или типич-

ная ошибка.

Заметка – дополнительная информация для углублённого понимания.

Итоги главы

В этой главе мы рассмотрели:

- Историю Anthropic и эволюцию линейки Claude
- Место Claude Code среди AI-инструментов для разработки
- Подробное сравнение с GitHub Copilot, Cursor, Q Developer и Gemini Code Assist
- Философию Claude Code: AI как напарник, а не замена
- Основные сценарии использования: от прототипирования до DevOps
- Объективные ограничения и реалистичные ожидания
- Структуру книги и условные обозначения

В следующей главе мы перейдём к практике: установим Claude Code, настроим окружение и сделаем первые шаги.

Глава 2. Установка и настройка окружения

2.1. Системные требования

Прежде чем приступить к установке Claude Code, убедимся, что ваша система соответствует минимальным требованиям. Claude Code – это инструмент командной строки, построенный на платформе Node.js, что обеспечивает его кроссплатформенность, но накладывает определённые ограничения.

Аппаратные требования

Claude Code сам по себе не требует мощного оборудования – все вычисления происходят на серверах Anthropic. Однако комфортная работа предполагает:

- **Оперативная память:** минимум 4 ГБ (рекомендуется 8 ГБ и более). Claude Code использует память для хранения контекста диалога и индексов проекта.

- **Дисковое пространство:** около 500 МБ для самого инструмента и его зависимостей, плюс место для ваших проектов.
- **Процессор:** любой современный процессор. CPU не является узким местом, поскольку основные вычисления выполняются в облаке.
- **Интернет-соединение:** стабильное подключение обязательно. Без интернета Claude Code работать не будет, так как все запросы к модели отправляются через API. Рекомендованная скорость – не менее 10 Мбит/с для комфортной работы.

Программные требования

- **Node.js:** версия 18.0 или выше. Claude Code написан на TypeScript и выполняется в среде Node.js. Рекомендуются использовать LTS-версию (Long Term Support) для максимальной стабильности.
- **npm:** версия 9.0 или выше. Поставляется вместе с Node.js.
- **Операционная система:**
 - macOS 12 (Monterey) или новее
 - Ubuntu 20.04 / Debian 11 или новее
 - Windows 10/11 с WSL2 (Windows Subsystem for Linux)
 - Другие дистрибутивы Linux с glibc 2.31+

- **Терминал:** любой эмулятор терминала. Рекомендуются: iTerm2 (macOS), Windows Terminal (Windows), Alacritty, Kitty или WezTerm (кроссплатформенные).
- **Git:** версия 2.30 или выше (для работы с контролем версий).

Проверка требований

Перед установкой выполните следующие проверки в терминале:

```
node -version
npm -version
git -version
```

Если Node.js не установлен или версия устарела, рекомендуется использовать менеджер версий – nvm (Node Version Manager):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install -lts
nvm use -lts
```

Для Windows рекомендуется nvm-windows:

```
winget install CoreyButler.NVMforWindows  
nvm install lts  
nvm use lts
```

2.2. Установка Claude Code

Установка через npm (рекомендуемый способ)

Основной и рекомендуемый способ установки Claude Code – через npm, менеджер пакетов Node.js:

```
npm install -g @anthropic-ai/claude-code  
claude -version
```

Глобальная установка (`-g`) делает команду `claude` доступной из любого каталога в терминале. Это важно, поскольку Claude Code работает в контексте текущего каталога – того проекта, с которым вы хотите работать.

Если вы столкнулись с ошибками прав доступа при глобальной установке на Linux/macOS, есть несколько решений:

```
sudo npm install -g @anthropic-ai/claude-code
mkdir -p ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:
$PATH' >> ~/.bashrc
source ~/.bashrc
npm install -g @anthropic-ai/claude-code
npx @anthropic-ai/claude-code
```

Установка через Homebrew (macOS)

На macOS доступна установка через Homebrew:

```
brew install claude-code
brew tap anthropic-ai/tap
brew install anthropic-ai/tap/claude-code
```

Установка на Windows через WSL2

Claude Code официально поддерживает Windows только

через WSL2. Вот пошаговая инструкция:

```
wsl -install
sudo apt update && sudo apt upgrade -y
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install -lts
npm install -g @anthropic-ai/claude-code
```

Важная деталь: при работе в WSL2 убедитесь, что ваш проект находится внутри файловой системы WSL (например, /home/username/projects/), а не на диске Windows (/mnt/c/...). Работа с файлами на диске Windows через WSL значительно медленнее из-за трансляции файловых систем.

Установка в Docker

Для изолированной среды или CI/CD можно использовать Docker:

```
FROM node:20-slim
RUN npm install -g @anthropic-ai/claude-code
```

```
code
  WORKDIR /app
  COPY . .
  CMD ["claude", "-print", "Analyze this
project"]
```

Обновление

Claude Code активно развивается, и новые версии выходят регулярно. Для обновления:

```
npm update -g @anthropic-ai/claude-code
npm install -g @anthropic-ai/claude-
code@latest
```

Рекомендуется обновляться регулярно – как минимум раз в две недели. Каждое обновление может содержать улучшения качества, новые инструменты и исправления ошибок.

2.3. Аутентификация и API-ключи

Claude Code взаимодействует с моделями Anthropic через API, для чего необходим ключ аутентификации. Существует

несколько способов настроить доступ.

Способ 1: Прямой вход через Anthropic

При первом запуске Claude Code предложит авторизоваться через веб-браузер:

```
claude
```

Откройте предложенную ссылку в браузере, войдите в свой аккаунт Anthropic и подтвердите авторизацию. После этого Claude Code сохранит токен авторизации локально и будет использовать его для последующих сессий.

Этот способ наиболее удобен для индивидуального использования: вам не нужно вручную управлять ключами, а токен автоматически обновляется.

Способ 2: API-ключ через переменную окружения

Для автоматизации, CI/CD или серверного использования удобнее задавать API-ключ через переменную окружения:

```
export ANTHROPIC_API_KEY="sk-ant-..."
echo 'export ANTHROPIC_API_KEY="sk-ant-...' >> ~/.bashrc
source ~/.bashrc
```

Для более безопасного хранения ключей используйте менеджер секретов:

```
echo 'export ANTHROPIC_API_KEY="sk-ant-...' > .envrc
direnv allow
export ANTHROPIC_API_KEY=$(op read "op://Development/Anthropic API Key/credential")
aws-vault exec dev - claude
```

Никогда не храните API-ключи в файлах, которые могут попасть в Git-репозиторий. Добавьте `.envrc` в `.gitignore`.

Способ 3: AWS Bedrock

Для организаций, использующих AWS, Claude доступен через Amazon Bedrock:

```
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export AWS_ACCESS_KEY_ID="AKIA..."
export AWS_SECRET_ACCESS_KEY="..."
```

Преимущества использования Bedrock: единый биллинг через AWS, SLA на корпоративном уровне, соответствие требованиям compliance (SOC2, HIPAA, FedRAMP).

Способ 4: Google Vertex AI

Аналогично, Claude доступен через Google Cloud Vertex AI:

```
export CLAUDE_CODE_USE_VERTEX=1
export CLOUD_ML_REGION=us-east5
export
ANTHROPIC_VERTEX_PROJECT_ID="your-
project-id"
```

Проверка аутентификации

После настройки убедитесь, что всё работает:

```
claude -print "Hello, world!"
```

Если возникает ошибка аутентификации, проверьте:

- Правильность API-ключа
- Наличие достаточного баланса на аккаунте
- Доступность серверов API (status.anthropic.com)
- Корректность сетевых настроек (прокси, VPN, файрвол)

2.4. Конфигурационные файлы

Claude Code использует систему конфигурационных файлов для настройки поведения. Понимание этой системы критически важно для эффективной работы.

Иерархия конфигурации

Claude Code применяет иерархическую систему настроек

с несколькими уровнями приоритета:

- **Аргументы командной строки** – высший приоритет
- **Переменные окружения**
- **Проектная конфигурация** (`.claude/` в каталоге проекта)
- **Пользовательская конфигурация** (`~/.claude/`)
- **Системные настройки по умолчанию** – низший приоритет

Файл CLAUDE.md

Файл `CLAUDE.md` – один из самых мощных инструментов настройки Claude Code. Это файл в формате Markdown, расположенный в корне проекта (или в любом подкаталоге для более granулярной настройки). Claude Code автоматически читает его и использует содержимое как постоянные инструкции.

Пример `CLAUDE.md` для типичного Python-проекта:

```
## Описание  
REST API для приложения управления задачами. Python 3.12, FastAPI, SQLAlchemy, PostgreSQL.
```

Архитектура

- `app/` - основной код приложения
- `api/` - маршруты и эндпоинты
- `models/` - SQLAlchemy модели
- `schemas/` - Pydantic схемы
- `services/` - бизнес-логика
- `core/` - конфигурация, безопасность,

зависимости

- `tests/` - тесты (pytest)
- `alembic/` - миграции базы данных

Соглашения по коду

- Используй type hints везде
- Используй async/await для всех эндпо-

интов

- Именованное: snake_case для функций, PascalCase для классов
- Формат строк: f-strings
- Максимальная длина строки: 100 символов
- Используй ruff для форматирования и

линтинга

Команды

- Запуск: `uvicorn app.main:app -reload`
- Тесты: `pytest -v`
- Линтинг: `ruff check .`
- Миграции: `alembic upgrade head`

Важно

- Не используй `print()` - всегда `logging`
- Все endpoints должны иметь описание и теги для OpenAPI
- Бизнес-логика НЕ должна быть в маршрутах - выноси в `services`
- Пароли хэшировать через `bcrypt`
- JWT-токены со сроком жизни 30 минут

Claude Code будет следовать этим инструкциям во всех своих действиях. Это как README для вашего AI-напарника – чем подробнее вы опишете проект, тем лучше Claude Code будет с ним работать.

Каталог `.claude/`

Каталог `.claude/` в корне проекта содержит дополнительные конфигурационные файлы:

```
.claude/  
+- settings.json # Настройки проекта  
+- commands/ # Пользовательские команды  
| +- review.md # /project:review  
| +- deploy.md # /project:deploy  
+- .cursorrules # Совместимость с Cursor
```

Файл `settings.json` управляет поведением Claude Code на уровне проекта:

```
{
  "permissions": {
    "allow": [
      "Read",
      "Write",
      "Bash(npm run *)",
      "Bash(pytest *)",
      "Bash(git *)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(sudo *)",
      "Bash(curl * | bash)"
    ]
  },
  "model": "claude-sonnet-4-20250514",
  "maxTurns": 50
}
```

Пользовательская конфигурация (~/.claude/)

Домашний каталог содержит глобальные настройки:


```
~/.claude/  
+- settings.json # Глобальные настройки  
+- commands/ # Глобальные команды  
| +- standup.md # /user:standup  
+- credentials.json # Токены авторизации  
(не редактировать)  
+- projects/ # Кэш и состояние проектов
```

Глобальные настройки применяются ко всем проектам и могут быть переопределены проектными:

```
{  
  "theme": "dark",  
  "verbose": false,  
  "autoApprove": ["Read"],  
  "notifications": true,  
  "defaultModel": "claude-  
sonnet-4-20250514"  
}
```

Переменные окружения

Claude Code поддерживает множество переменных окружения для тонкой настройки:

ANTHROPIC_API_KEY # API-ключ Anthropic
CLAUDE_CODE_USE_BEDROCK # Использовать
AWS Bedrock
CLAUDE_CODE_USE_VERTEX # Использовать
Google Vertex AI
CLAUDE_MODEL # Модель по умолчанию
HTTP_PROXY # HTTP прокси
HTTPS_PROXY # HTTPS прокси
NO_PROXY # Исключения для прокси
CLAUDE_CODE_MAX_TURNS # Максимальное ко-
личество шагов
CLAUDE_CODE_DISABLE_TELEMETRY # Отключе-
ние телеметрии

2.5. Интеграция с редакторами кода

Хотя Claude Code работает из терминала и не зависит от конкретного редактора, существуют интеграции, которые делают работу более удобной.

Visual Studio Code

VS Code – самый популярный редактор среди разработчиков, и для него доступно расширение Claude Code:

```
code -install-extension anthropic.claude-code-vscode
```

Расширение добавляет:

- Панель Claude Code в боковой панели
- Возможность отправлять выделенный код в Claude Code
- Отображение изменений, внесённых Claude Code, в diff-просмотрщике
- Подсветку файла CLAUDE.md
- Статус-бар с информацией о текущей сессии

Альтернативный подход – использовать встроенный терминал VS Code. Откройте терминал (`Ctrl+`` или `Cmd+``), перейдите в каталог проекта и запустите `claude``. Оба окна – редактор и Claude Code – будут работать бок о бок, и вы сможете видеть изменения в файлах в реальном времени.

Полезные настройки VS Code для работы с Claude Code:

```
// settings.json
{
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  "editor.formatOnSave": true,
  "git.autofetch": true,
  "terminal.integrated.scrollback": 10000
}
```

JetBrains IDE (IntelliJ IDEA, PyCharm, WebStorm)

Для семейства JetBrains IDE существует плагин Claude Code:

- Откройте Settings -> Plugins -> Marketplace
- Найдите «Claude Code»
- Установите и перезапустите IDE

Плагин интегрируется в панель инструментов и позволяет:

- Отправлять выделенный код или файл в Claude Code
- Просматривать изменения через встроенный diff-просмотрщик JetBrains

– Использовать Claude Code из встроенного терминала

Как и в случае с VS Code, самый простой способ – открыть встроенный терминал и запустить `claude` из каталога проекта.

Vim / Neovim

Для пользователей Vim/Neovim рекомендуется работать с Claude Code через разделение терминала (`tmux`, `screen` или встроенная функция терминального мультиплексора):

```
tmux new-session -d -s dev
tmux split-window -h
tmux send-keys -t dev:0.0 'vim .' Enter
tmux send-keys -t dev:0.1 'claude' Enter
tmux attach -t dev
```

Для Neovim существуют плагины интеграции:

```
-- lazy.nvim
{
  "anthropic/claude-code.nvim",
  config = function()
    require("claude-code").setup({
```

```
auto_open = false,  
split = "vertical",  
)  
end,  
)
```

Emacs

Для Emacs доступен пакет `claude-code.el`:

```
;; Через use-package  
(use-package claudе-code  
:ensure t  
:config  
(setq claudе-code-auto-approve-reads t))
```

Или просто используйте `M-x shell` или `M-x vterm` для запуска Claude Code в буфере терминала внутри Emacs.

2.6. Настройка рабочего окружения

Для максимально продуктивной работы с Claude Code рекомендуется настроить ряд дополнительных инструментов.

Терминальный мультиплексор

tmux или screen позволяют работать с несколькими панелями терминала одновременно:

```
brew install tmux
sudo apt install tmux
set -g mouse on
set -g history-limit 50000
set -g default-terminal
"screen-256color"
bind | split-window -h
bind - split-window -v
```

Типичная раскладка для работы с Claude Code:

- Левая панель: редактор кода (Vim/Neovim)
- Правая верхняя панель: Claude Code
- Правая нижняя панель: терминал для запуска команд вручную

Инструменты для работы с файлами

Claude Code активно работает с файлами, и полезно иметь инструменты для их просмотра:

```
brew install bat
sudo apt install bat
brew install fd # или apt install fd-find
brew install ripgrep # или apt install
ripgrep
brew install fzf # или apt install fzf
brew install eza # или cargo install eza
brew install git-delta # или cargo
install git-delta
```

Настройка Git для работы с delta:

```
[core]
pager = delta
[interactive]
diffFilter = delta -color-only
[delta]
navigate = true
side-by-side = true
line-numbers = true
```


Настройка Shell

Удобная оболочка ускоряет работу:

```
sh -c "$ (curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh) "
plugins=(
git
docker
kubectl
node
npm
python
fzf
zsh-autosuggestions
zsh-syntax-highlighting
)
alias cc='claude'
alias ccp='claude -print'
alias ccr='claude -resume'
alias ccc='claude -continue'
```

2.7. Первый запуск и ориентация

После установки и настройки давайте выполним первый запуск и познакомимся с интерфейсом.

Запуск

Откройте терминал, перейдите в каталог проекта и запустите Claude Code:

```
cd ~/projects/my-project
claude
```

При первом запуске вы увидите приветственное сообщение и приглашение ввести запрос:

```
+=====
+
| Welcome to Claude Code! |
| |
| Model: claude-sonnet-4-20250514 |
| Context: 200,000 tokens |
| Working directory: ~/projects/my-
```

```
project |
+=====
+
Type your request or /help for commands.
>
```

Основные команды

Claude Code поддерживает набор слэш-команд для управления:

- /help - Справка по командам
- /clear - Очистка диалога и сброс контекста
- /compact - Сжатие контекста для экономии токенов
- /status - Информация о текущей сессии
- /cost - Стоимость текущей сессии
- /model - Смена модели
- /permissions - Управление разрешениями
- /config - Настройки
- /bug - Отправка отчёта об ошибке
- /quit или Ctrl+C - Выход

Проверка работоспособности

Попробуйте простой запрос:

```
> Покажи список файлов в текущем каталоге  
и опиши структуру проекта  
Claude Code выполнит команду ls или find,  
прочитает ключевые файлы  
(package.json, requirements.txt,  
README.md и т.п.)  
и выдаст описание структуры проекта.
```

Если Claude Code успешно выводит ответ, значит установка прошла корректно.

2.8. Типичные проблемы при установке и их решение

Ошибка «command not found: claude»

Наиболее частая проблема. Причины и решения:

```
npm list -g @anthropic-ai/claude-code  
echo $PATH  
export PATH=$(npm config get prefix)/bin:  
$PATH
```

Ошибка «EACCES: permission denied»

```
mkdir -p ~/.npm-global  
npm config set prefix '~/.npm-global'  
export PATH=~/.npm-global/bin:$PATH  
npm install -lts
```

Ошибка аутентификации (401 Unauthorized)

```
echo $ANTHROPIC_API_KEY  
claude logout  
claude login
```

Ошибка «rate limit exceeded»

Anthropic ограничивает количество запросов для защиты от злоупотреблений. Решения:

- Подождать несколько минут и повторить
- Перейти на более высокий тарифный план
- Использовать `/compact` для уменьшения размера запросов
- Разбить задачу на более мелкие части

Проблемы с прокси/VPN

```
export HTTP_PROXY=http://  
proxy.company.com:8080  
export HTTPS_PROXY=http://  
proxy.company.com:8080  
export NO_PROXY=localhost,127.0.0.1  
export NODE_EXTRA_CA_CERTS=/path/to/  
corporate-ca.crt
```

Проблемы на Windows/WSL

```
wsl -list -verbose  
wsl -set-version Ubuntu 2  
nslookup api.anthropic.com  
echo "nameserver 8.8.8.8" | sudo tee /
```

2.9. Управление затратами

Claude Code использует API-тарификацию Anthropic, и понимание модели ценообразования поможет контролировать расходы.

Модель ценообразования

Стоимость рассчитывается за количество токенов:

- **Входные токены** (ваш промпт + контекст): тарифицируются по одной ставке
- **Выходные токены** (ответ Claude): тарифицируются по более высокой ставке
- **Кэшированные токены**: тарифицируются со скидкой (если контекст не изменился)

Цены различаются в зависимости от модели:

- Claude Sonnet – оптимальный баланс цены и качества
- Claude Opus – дороже, но лучше для сложных задач

– Claude Haiku – дешевле, подходит для простых задач

Стратегии экономии

– **Используйте /compact регулярно.** Эта команда сжимает историю диалога, сохраняя ключевую информацию и уменьшая количество входных токенов.

– **Начинайте с Sonnet.** В большинстве случаев Sonnet обеспечивает достаточное качество. Переключайтесь на Opus только для особо сложных задач.

– **Будьте конкретны.** Чем точнее ваш промпт, тем меньше итераций потребуется. Вместо «исправь баги» скажите «в файле auth.py функция login_user возвращает 500 при невалидном email, а должна возвращать 422 с подробным описанием ошибки».

– **Используйте CLAUDE.md.** Предоставление контекста через CLAUDE.md снижает необходимость в повторных объяснениях и уменьшает количество операций чтения файлов.

– **Разбивайте задачи.** Одна большая задача может потребовать загрузки всей кодовой базы в контекст. Несколько

мелких задач работают с меньшим контекстом.

– **Мониторьте расходы.** Используйте команду `/cost` для отслеживания расходов текущей сессии и панель управления на console.anthropic.com для общей статистики.

Бюджетирование

Для оценки бюджета учитывайте:

- Активный разработчик тратит в среднем 10-50 долларов в месяц
- Интенсивное использование (целый рабочий день каждый день) может достигать 100-200 долларов
- Для команд удобнее использовать централизованный биллинг через AWS Bedrock или Google Vertex

Итоги главы

В этой главе мы:

- Проверили системные требования для Claude Code
- Установили Claude Code через npm, Homebrew или в

Docker

- Настроили аутентификацию через веб-вход, API-ключ или облачного провайдера
- Изучили систему конфигурационных файлов: CLAUDE.md, .claude/, ~/.claude/
- Настроили интеграцию с VS Code, JetBrains, Vim/Neovim и Emacs
- Оптимизировали рабочее окружение: tmux, утилиты командной строки, алиасы
- Разобрали типичные проблемы при установке и их решения
- Научились управлять затратами на API

В следующей главе мы сделаем первые практические шаги: создадим проект, научимся основным паттернам взаимодействия и напишем первый код с помощью Claude Code.

Глава 3. Первые шаги с Claude Code

3.1. Запуск и интерфейс командной строки

Итак, Claude Code установлен, аутентификация настроена, и вы готовы к работе. Откройте терминал и перейдите в каталог с проектом – или созданием новый:

```
mkdir -p ~/projects/my-first-project  
cd ~/projects/my-first-project  
claude
```

Claude Code запускается и представляет лаконичный интерфейс командной строки. В отличие от громоздких графических приложений, здесь нет кнопок, меню или панелей инструментов – только текстовое поле для ввода и область для отображения ответов. Эта минималистичность обманчива: за ней скрывается мощная система, способная выполнять сложнейшие операции.

Интерфейс Claude Code состоит из нескольких элементов:

Строка приглашения (prompt) – символ `>`, за которым

вы вводите свои запросы. Здесь вы общаетесь с AI на естественном языке: задаёте вопросы, ставите задачи, просите объяснить код.

Область вывода – пространство, где Claude Code отображает свои ответы, включая текст, код, визуализации прогресса и результаты выполнения инструментов.

Индикаторы инструментов – при выполнении операций (чтение файлов, запуск команд, запись в файлы) Claude Code показывает, какой инструмент используется и с какими параметрами. Это обеспечивает прозрачность: вы всегда видите, что именно делает AI.

Строка состояния – внизу экрана отображается информация о текущей модели, стоимости сессии и количестве использованных токенов.

Режимы работы

Claude Code поддерживает несколько режимов работы:

Интерактивный режим (по умолчанию) – вы ведёте диалог с Claude Code, поочерёдно отправляя запросы и получая ответы. Это основной режим для повседневной рабо-

ты.

```
claude # Запуск в интерактивном режиме
```

Однократный режим (print mode) – Claude Code обрабатывает один запрос и завершает работу. Удобно для скриптов и автоматизации.

```
claude -print "Объясни, что делает файл  
main.py"
```

Режим продолжения – возобновление предыдущего диалога:

```
claude -continue # Продолжить последний  
диалог
```

```
claude -resume # Выбрать из списка преды-  
дущих диалогов
```

Pipe-режим – Claude Code принимает входные данные через стандартный ввод:

```
cat error.log | claude -print "Проанали-  
зируй эти логи ошибок"
```

```
git diff | claude -print "Проведи code  
review этих изменений"
```

Pipe-режим – невероятно мощная возможность, потому что он позволяет встраивать Claude Code в любой конвейер Unix-команд. Представьте:

```
grep -rn "TODO:" src/ | claude -print  
"Расставь приоритеты для этих задач"  
git log --oneline -20 | claude -print "На-  
пиши changelog из этих коммитов"  
npm audit --json | claude -print "Объясни  
найденные уязвимости и предложи решения"
```

3.2. Анатомия диалога

Каждое взаимодействие с Claude Code можно разложить на несколько фаз. Понимание этих фаз поможет строить более эффективные диалоги.

Фаза 1: Получение запроса

Вы вводите запрос на естественном языке. Claude Code принимает запрос, добавляет к нему системный контекст (содержимое CLAUDE.md, информацию о проекте, историю

диалога) и отправляет на обработку.

Фаза 2: Планирование

Claude Code анализирует запрос и определяет, какие действия необходимы для его выполнения. Для сложных задач он может сформулировать явный план:

> Добавь аутентификацию через JWT в наш API

Claude Code:

Для добавления JWT-аутентификации мне потребуется:

1. Установить необходимые зависимости (PyJWT, passlib)
 2. Создать модель User
 3. Написать функции хеширования паролей
 4. Реализовать endpoints регистрации и входа
 5. Создать middleware для проверки токенов
 6. Обновить существующие endpoints
- Начинаю с зависимостей...

Фаза 3: Выполнение инструментов

Claude Code последовательно вызывает инструменты для реализации плана. Каждый вызов отображается в интерфейсе:

```
Tool: ReadFile
File: requirements.txt
Tool: Bash
Command:      pip      install      PyJWT
passlib[bcrypt]
Tool: WriteFile
File: app/models/user.py
[показ содержимого файла]
Tool: WriteFile
File: app/core/security.py
[показ содержимого файла]
```

Фаза 4: Запрос подтверждения (при необходимости)

Для потенциально опасных операций Claude Code запрашивает ваше подтверждение:

Tool: Bash

Command: pip install PyJWT

passlib[bcrypt]

Allow? [Y/n/always]

Варианты ответа:

– Y или Enter – разрешить выполнение

– n – отклонить

– always – разрешить и запомнить для этого типа операций

Фаза 5: Подведение итогов

После выполнения всех шагов Claude Code подводит итог:

Готово! Я добавил JWT-аутентификацию:

[+] Установлены PyJWT и passlib

[+] Создана модель User с хешированием пароля

[+] Реализованы эндпоинты /auth/register и /auth/login

[+] Создан middleware verify_token

[+] Обновлено защищённые маршруты

Для тестирования:

```
curl -X POST http://localhost:8000/auth/register \
-H "Content-Type: application/json" \
-d '{"email": "user@example.com", "password": "secret123"}'
```

3.3. Базовые команды и навигация

Слэш-команды

Claude Code поддерживает набор слэш-команд для управления сессией. Разберём каждую подробно.

/help – Справка

```
> /help
```

Доступные команды:

/help Показать эту справку

/clear Очистить контекст диалога

/compact Сжать контекст (экономия токенов)

/status Информация о сессии

`/cost` Стоимость текущей сессии
`/model` Переключить модель
`/permissions` Управление разрешениями
`/config` Настройки
`/bug` Отправить отчёт об ошибке
`/quit` Выход

`/clear` – Сброс контекста

Команда `/clear` полностью сбрасывает контекст диалога. Используйте её, когда:

- Переключаетесь на совершенно другую задачу
- Контекст стал слишком большим и замедляет работу
- Claude Code «запутался» и даёт неадекватные ответы

После `/clear` Claude Code забудет всё, что обсуждалось ранее, но сохранит конфигурацию проекта (CLAUDE.md, settings и т. д.).

`/compact` – Сжатие контекста

`/compact` – одна из наиболее полезных команд. Она заставляет Claude Code сжать историю диалога, сохранив ключевую информацию, но уменьшив объём контекста. Это экономит токены и деньги.

```
> /compact
```

Контекст сжат:

До: 45 832 токена

После: 12 447 токенов

Сэкономлено: 33 385 токенов (~73%)

Рекомендуется вызывать `/compact` после каждого крупного блока работы – например, после завершения реализации конкретной функции.

Вы также можете указать фокус при сжатии:

```
> /compact Сфокусируйся на работе с базой данных
```

Claude Code сожмёт контекст, но сохранит детали, связанные с базой данных, и удалит менее релевантную информацию.

`/status` – Информация о сессии

```
> /status
```

Session Info:

Model: claude-sonnet-4-20250514

Context used: 23,451 / 200,000 tokens

(12%)

Messages: 14

Files read: 8

Files written: 3

Commands executed: 5

Duration: 12m 34s

Cost: \$0.47

/cost – Детализация стоимости

> /cost

Cost Breakdown:

Input tokens: 187,234 (\$0.32)

Output tokens: 41,892 (\$0.15)

Cached tokens: 124,891 (\$0.04)

Total: \$0.47

Горячие клавиши

Claude Code поддерживает стандартные горячие клавиши терминала:

- Ctrl+C – отменить текущую операцию или выйти
- Ctrl+D – выйти из Claude Code
- Up/Down – навигация по истории запросов

- Tab – автодополнение слэш-команд
- Ctrl+L – очистить экран (без сброса контекста)
- Esc – отменить ввод текущего промпта

Многострочный ввод

Для длинных промптов используйте многострочный ввод. Нажмите \ в конце строки для продолжения на следующей, или используйте Shift+Enter (в поддерживающих терминалах):

```
> Создай REST API со следующими эндпоин-  
тами: \  
GET /users – список пользователей, \  
POST /users – создание пользователя, \  
GET /users/:id – получение пользователя,  
\  
PUT /users/:id – обновление, \  
DELETE /users/:id – удаление. \  
Используй Express.js и MongoDB.
```

3.4. Управление контекстным окном

Контекстное окно – это «рабочая память» Claude Code. Всё, что вы обсудили, все прочитанные файлы и результаты выполненных команд хранятся в этом окне. Когда окно заполняется, старая информация начинает вытесняться, что может привести к потере контекста.

Размер контекстного окна

В зависимости от модели, контекстное окно Claude Code составляет:

- Claude Sonnet: ~200 000 токенов
- Claude Opus: ~200 000 токенов
- Claude Haiku: ~200 000 токенов

Один токен – это примерно 4 символа для английского текста и 1-2 символа для русского. Файл на 1000 строк занимает примерно 10 000-20 000 токенов.

Стратегии управления контекстом

1. Будьте избирательны. Не просите Claude Code «прочитай все файлы в проекте». Вместо этого укажите конкретные файлы или каталоги:

```
> Проанализируй весь проект  
> Проанализируй файлы в каталоге src/auth/ – мне нужно  
понять текущую систему аутентификации
```

2. Используйте /compact регулярно. Как упоминалось ранее, сжатие контекста высвобождает место для новой информации.

3. Разбивайте задачи. Вместо одного длинного диалога ведите несколько коротких. Завершили реализацию одной функции – начните новый диалог для следующей.

4. Используйте CLAUDE.md. Информация из CLAUDE.md загружается автоматически и занимает меньше места, чем повторные объяснения в диалоге.

5. Следите за индикатором. Команда /status показывает, какой процент контекстного окна использован. При

приближении к 70-80% подумайте об оптимизации.

Что происходит при переполнении

Когда контекстное окно заполняется, Claude Code применяет алгоритм сжатия:

- Старые сообщения сжимаются в краткие резюме
- Результаты выполненных команд усекаются
- Содержимое прочитанных файлов может быть частично удалено
- Системные промпты и CLAUDE.md сохраняются в приоритете

Вы можете заметить, что Claude Code «забыл» что-то, что обсуждалось ранее. Это нормально – просто напомните ему нужный контекст.

3.5. Первый проект: создание веб-приложения

Давайте создадим полноценный проект, чтобы увидеть Claude Code в действии. Мы построим простое, но функциональное веб-приложение – менеджер заметок.

Шаг 1: Инициализация проекта

> Создай новый проект – менеджер заметок.
Стек: Node.js, Express, SQLite, EJS для шаблонов.

Базовая функциональность: создание, чтение, редактирование, удаление заметок. Каждая заметка имеет заголовок, содержимое и дату создания/обновления.

Claude Code начнёт работу:

- Создаст `package.json` с зависимостями
- Инициализирует npm-проект
- Создаст структуру каталогов:

```
notes-app/  
+- package.json  
+- server.js  
+- database.js  
+- routes/  
  | +- notes.js  
+- views/
```

```
| +- layout.ejs  
| +- index.ejs  
| +- note.ejs  
| +- edit.ejs  
+- public/  
| +- style.css  
+- README.md
```

- Напишет код сервера, маршруты, модели и шаблоны
- Установит зависимости
- Проверит работоспособность

Шаг 2: Проверка и запуск

После генерации проверим, что всё работает:

> Запусти приложение и проверь, что главная страница открывается

Claude Code выполнит:

```
node server.js
```

И проверит доступность через curl или встроенный инструмент.

Шаг 3: Доработка

Теперь добавим функциональность:

> Добавь поиск по заметкам: поле поиска на главной странице, которое фильтрует заметки по заголовку и содержимому.

Поиск должен быть нечувствительным к регистру.

Claude Code изменит несколько файлов:

- Добавит маршрут `/search` в `routes/notes.js`
- Добавит SQL-запрос с `LIKE` в `database.js`
- Обновит шаблон `index.ejs`, добавив форму поиска
- Обновит стили в `style.css`

Шаг 4: Итерация

Этот процесс можно продолжать сколько угодно:

- > Добавь категории для заметок
- > Добавь markdown-рендеринг в содержимом заметки
- > Добавь экспорт заметок в PDF
- > Добавь тёмную тему

Каждый запрос – это итерация, которая наращивает функциональность проекта. Claude Code помнит контекст предыдущих шагов и строит новые функции на существующей основе.

3.6. Работа с файлами

Работа с файлами – одна из основных возможностей Claude Code. Разберём ключевые операции.

Чтение файлов

Claude Code может читать файлы различными способами:

- > Покажи содержимое файла `server.js`
- > Прочитай все файлы в каталоге `src/`
- > Что находится в `package.json`?

При чтении файла Claude Code отображает его содержимое и подтверждает, что файл загружен в контекст. Это позволяет модели анализировать код и предлагать изменения.

Создание файлов

> Создай файл `config.js` с настройками приложения:

порт, строка подключения к БД, секретный ключ для JWT

Claude Code создаст файл с осмысленным содержимым:

```
// config.js
const config = {
  port: process.env.PORT || 3000,
  database: {
    url: process.env.DATABASE_URL ||
'sqlite:./data/app.db',
  },
  jwt: {
    secret: process.env.JWT_SECRET ||
'change-me-in-production',
    expiresIn: '24h',
```

```
},  
app: {  
  name: 'Notes Manager',  
  version: '1.0.0',  
},  
};  
module.exports = config;
```

Редактирование файлов

Claude Code может вносить точечные изменения, не пере-записывая весь файл:

```
> В файле server.js добавь middleware для  
логирования  
запросов с помощью morgan
```

Claude Code прочитает файл, определит нужное место для вставки и внесёт минимальные изменения:

```
// Добавлено:  
const morgan = require('morgan');  
// ...  
app.use(morgan('dev'));
```

Удаление файлов

> Удали файл `temp.js` – он больше не нужен

Claude Code запросит подтверждение перед удалением.

Переименование и перемещение

> Переименуй `database.js` в `db/connection.js`

Claude Code создаст каталог, переместит файл и обновит все `import/require`-ссылки в проекте.

Массовые операции

Одно из преимуществ агентного подхода – способность выполнять массовые операции:

> Добавь JSDoc-комментарии ко всем экспортируемым функциям
в каталоге `src/utils/`

Claude Code последовательно прочитает каждый файл, проанализирует функции, добавит документацию и сохранит изменения.

3.7. Работа с терминалом

Claude Code может выполнять команды в терминале – это даёт ему возможность непосредственно взаимодействовать с операционной системой, инструментами разработки и инфраструктурой.

Безопасность выполнения команд

Прежде чем обсуждать возможности, важно подчеркнуть: Claude Code не выполняет произвольные команды без вашего ведома. Система разрешений работает следующим образом:

- **Безопасные команды** (чтение файлов, `ls`, `cat`, `find`, `grep`) – выполняются без подтверждения
- **Команды разработки** (`npm run`, `pytest`, `go build`) – зависят от настроек

- **Потенциально опасные** (установка пакетов, изменение конфигурации) – требуют подтверждения
- **Опасные** (rm, sudo, curl | bash) – всегда требуют подтверждения

Вы можете настроить уровень доверия через разрешения в `.claude/settings.json`.

Типичные сценарии выполнения команд

Установка зависимостей:

```
> Установи необходимые зависимости для  
проекта  
Claude Code: Для этого мне нужно выпол-  
нить:  
npm install express ejs better-sqlite3  
morgan  
Allow? [Y/n/always]
```

Запуск тестов:

```
> Запусти тесты и исправь падающие  
Claude Code выполнит `npm test`, проана-  
лизирует вывод,
```

найдёт причины ошибок и исправит код.

Сборка проекта:

> Собери проект для продакшена
Claude Code выполнит ``npm run build``,
проверит результат,
исправит проблемы при необходимости.

Работа с Git:

> Создай коммит с осмысленным сообщением
Claude Code проанализирует diff, сформу-
лирует сообщение
и выполнит ``git add`` + ``git commit``.

Анализ системы:

> Покажи использование дискового про-
странства в проекте
> Какие процессы занимают порт 3000?
> Проверь, запущен ли Docker

Конвейеры команд

Claude Code может строить сложные конвейеры команд и анализировать их результаты:

```
> Найди все TypeScript файлы с неиспользуемыми импортами
```

```
Claude Code может выполнить:
```

```
npx ts-prune | head -50
```

И затем проанализировать результат, показав вам список

файлов с неиспользуемыми экспортами.

3.8. Навигация по проекту

Одна из мощных возможностей Claude Code – способность самостоятельно навигировать по проекту. Вы не обязаны указывать точные имена файлов и пути – Claude Code найдёт нужное самостоятельно.

Поиск файлов

> Где определена функция calculateDiscount?

Claude Code использует grep/ripgrep для поиска:

```
rg "function calculateDiscount|  
calculateDiscount =" src/
```

И найдёт: src/utils/pricing.js:42

Понимание зависимостей

> Какие файлы импортируют модуль database.js?

Claude Code найдёт все файлы, которые используют

```
require('./database') или import from './  
database'.
```

Анализ структуры

> Опиши архитектуру проекта

Claude Code прочитает ключевые файлы (package.json, структуру каталогов, основные модули) и составит описание архитектуры.

Поиск по содержимому

> Найди все места, где используется deprecated API из библиотеки axios
Claude Code выполнит поиск по шаблонам и покажет результаты с контекстом.

3.9. Мультимодальность: работа с изображениями

Claude Code (через модели Claude) поддерживает мультимодальность – он может анализировать изображения. Это полезно в нескольких сценариях:

Реализация дизайна

Если у вас есть макет интерфейса в виде изображения, вы можете передать его Claude Code:

> Вот макет главной страницы [скриншот].
Реализуй этот дизайн в HTML/CSS.

Claude Code проанализирует изображение и создаст HTML/CSS-код, максимально приближенный к макету.

Анализ скриншотов ошибок

> Вот скриншот ошибки в браузере [изображение].

Что может быть причиной?

Claude Code прочитает текст ошибки с изображения и предложит решение.

Диаграммы и схемы

> Вот диаграмма архитектуры [изображение].

Создай Docker Compose файл на основе этой схемы.

3.10. Паттерны эффективного взаимодействия

За время работы с Claude Code сообщество выработало ряд паттернов, которые значительно повышают эффективность.

Паттерн «Контекст -> Задача -> Ограничения»

Структурируйте запросы в три блока:

> [Контекст] Мы разрабатываем e-commerce платформу на Django.

Сейчас работаем над модулем корзины покупок.

[Задача] Нужно реализовать функцию при-

менения промокодов.

Промокод может давать процентную скидку (10%, 20%) или

фиксированную (500 руб, 1000 руб). Один промокод на заказ.

[Ограничения] Используй существующую модель CartItem.

Промокоды хранятся в таблице Promo. Валидацию делай на уровне сервиса, не в модели.

Паттерн «Обзор -> Детализация -> Реализация»

Для сложных задач разбивайте работу на три этапа:

> Предложи архитектуру системы уведомлений.

Поддержка: email, push, SMS, in-app.

> Отлично, давай детализируем вариант с очередями.

Как будет выглядеть интерфейс NotificationService?

> Реализуй NotificationService согласно обсуждённому плану.

Паттерн «Покажи перед изменением»

Перед внесением изменений в критический код попросите Claude Code показать план:

```
> Покажи, как ты собираешься изменить  
auth middleware,  
но пока не меняй файлы. Я хочу посмотреть  
план.
```

Это даёт возможность скорректировать направление до того, как будут внесены изменения.

Паттерн «Исправь и проверь»

После каждого блока изменений просите запустить проверку:

```
> Исправь ошибку в парсере и запусти те-  
сты,  
чтобы убедиться, что ничего не сломалось.
```

Этот паттерн помогает ловить регрессии на ранней ста-

дии.

Паттерн «Объясни как учителю»

Когда вы хотите не просто получить код, но и понять решение:

> Реализуй алгоритм LRU-кэша и объясни каждый шаг, как будто я джуниор-разработчик. Почему именно такая структура данных? Какие альтернативы?

3.11. Управление сессиями

Claude Code поддерживает сохранение и возобновление диалогов, что позволяет продолжить работу с того места, где вы остановились.

Продолжение последнего диалога

```
claude -continue
```

```
claude -c
```

Это загрузит последний диалог со всей историей и контекстом. Полезно, когда вы закрыли терминал и хотите продолжить работу.

Возобновление конкретного диалога

```
claude -resume
```

Откроет список предыдущих диалогов с описаниями, из которого вы можете выбрать нужный.

Организация диалогов

Для организации работы рекомендуется:

- Один диалог – одна задача (фича, баг, рефакторинг)
- Используйте `/clear` перед началом совершенно новой

задачи

- Используйте `--continue` для продолжения незавершённой работы
- Делайте `/compact` перед сохранением длинного диалога

3.12. Практическое упражнение: от нуля до деплоя

Закрепим полученные знания комплексным упражнением. Мы создадим, настроим и подготовим к деплою веб-приложение за один диалог.

Задание

Создайте приложение «Книжная полка» – каталог книг с возможностью добавления, редактирования, удаления, поиска и фильтрации по жанрам.

Рекомендуемая последовательность промптов

1. > Создай приложение "Книжная полка" на

Express.js + SQLite + EJS.

Модель книги: название, автор, жанр, год, рейтинг, обложка (URL).

CRUD для книг, поиск, фильтрация по жанру.

Красивый, современный интерфейс.

2. > Запусти приложение и проверь, что всё работает.

3. > Добавь 10 тестовых книг из русской классики с реальными данными.

4. > Добейся, чтобы приложение красиво выглядело на мобильных устройствах.

Используй responsive дизайн.

5. > Добавь пагинацию: по 6 книг на страницу.

6. > Напиши тесты для API-эндпоинтов.

7. > Создай Dockerfile для деплоя.

8. > Напиши README.md с инструкциями по запуску и деплою.

После выполнения этого упражнения у вас будет полноценное приложение, готовое к развёртыванию, – и глубокое понимание рабочего процесса с Claude Code.

Итоги главы

В этой главе мы:

- Освоили запуск Claude Code и интерфейс командной строки
- Изучили режимы работы: интерактивный, однократный, продолжение и pipe
- Поняли анатомию диалога: от запроса через планирование и выполнение к итогам
- Разобрали базовые слэш-команды: /help, /clear, /compact, /status, /cost
- Научились управлять контекстным окном для оптимальной работы
- Создали первый проект – менеджер заметок
- Освоили работу с файлами: чтение, создание, редактирование, удаление
- Разобрали работу с терминалом и систему разрешений
- Изучили навигацию по проекту: поиск файлов, зависимостей, структуры
- Узнали о паттернах эффективного взаимодействия
- Научились управлять сессиями
- Выполнили практическое упражнение: приложение от нуля до деплоя

В следующей главе мы погрузимся глубже – в архитектуру и принципы работы Claude Code: как он понимает код, какие инструменты использует, и как устроена его система безопасности.

Глава 4. Архитектура и принципы работы

4.1. Как Claude Code понимает вашу кодовую базу

Когда вы запускаете Claude Code в каталоге проекта, происходит нечто большее, чем простое чтение файлов. Claude Code использует многоуровневую систему для построения понимания вашего проекта. Разберём каждый уровень этой системы.

Уровень 1: Метаданные проекта

Первое, что делает Claude Code при старте – анализирует метаданные проекта. Это файлы, которые описывают проект на высоком уровне:

- **package.json** / **pyproject.toml** / **go.mod** / **Cargo.toml** – определяет язык, зависимости, скрипты
- **README.md** – описание проекта, инструкции по за-

пуску

- **CLAUDE.md** – специальные инструкции для AI-ассистента
- **.gitignore** – какие файлы исключены из VCS (и, следовательно, менее важны)
- **Dockerfile / docker-compose.yml** – инфраструктура
- **tsconfig.json / eslint.config.js / ruff.toml** – конфигурация инструментов

Эти файлы дают Claude Code «карту местности» – общее представление о технологическом стеке, структуре и конвенциях проекта. Без них Claude Code всё равно может работать, но с ними – значительно эффективнее.

Уровень 2: Структура файловой системы

Claude Code анализирует иерархию каталогов проекта. Структура каталогов – это, по сути, архитектурная диаграмма проекта. Например, структура:

```
src/  
+- controllers/  
+- models/  
+- services/  
+- repositories/
```

```
+– middleware/  
+– utils/  
+– config/
```

Немедленно говорит Claude Code, что перед ним MVC-приложение с чёткой слоистой архитектурой. Это влияет на то, куда он поместит новый код и как будет именовать файлы.

Другой пример:

```
src/  
+– features/  
| +- auth/  
| | +- components/  
| | +- hooks/  
| | +- api/  
| | +- types/  
| +- dashboard/  
| +- settings/  
+– shared/
```

Здесь Claude Code распознаёт feature-based архитектуру и будет организовывать новый код по тому же принципу – группируя по функциональности, а не по типу файла.

Уровень 3: Индексация кода

При необходимости Claude Code просматривает содержимое файлов, строя ментальную модель кодовой базы. Он не загружает все файлы в контекст одновременно (это было бы слишком затратно), а использует стратегию «ленивой загрузки»:

- **Сначала** загружаются метаданные и структура
- **По запросу** загружаются конкретные файлы, релевантные текущей задаче
- **При поиске** используются инструменты вроде `grep` и `find` для локализации нужного кода
- **Перекрёстные ссылки** отслеживаются через анализ импортов и экспортов

Этот подход позволяет работать с проектами любого размера. Вам не нужен проект из десяти файлов – Claude Code успешно работает с монорепозитариями, содержащими тысячи файлов.

Уровень 4: Семантическое понимание

На этом уровне Claude Code использует возможности языковой модели для семантического понимания кода. Это включает:

- **Распознавание паттернов проектирования:** Singleton, Factory, Observer, Repository, Decorator и другие
- **Понимание бизнес-логики:** Claude Code не просто видит функцию `calculateTax`, а понимает, что она связана с налоговыми расчётами, и может предложить добавить обработку различных налоговых юрисдикций
- **Анализ потока данных:** как данные проходят через систему – от входа (API-запрос) через обработку (сервисный слой) до хранения (база данных)
- **Выявление потенциальных проблем:** race conditions, утечки памяти, SQL-инъекции, небезопасное обращение с секретами

4.2. Инструменты (Tools): как Claude Code действует

В основе агентных возможностей Claude Code лежит си-

стема инструментов (tools). Каждый инструмент – это атомарное действие, которое Claude Code может выполнить во внешней среде. Модель сама решает, какие инструменты вызывать, в каком порядке и с какими параметрами.

Файловые инструменты

ReadFile – чтение содержимого файла:

Input: path: "src/utils/helpers.ts"

Output: содержимое файла (текст)

Claude Code использует этот инструмент для загрузки файлов в контекст. При чтении больших файлов он может запросить только определённый диапазон строк.

WriteFile – создание или перезапись файла:

Input: path: "src/models/user.ts"

content: "export interface User { ... }"

Output: файл создан/обновлён

EditFile (MultiEdit) – точечное редактирование файла:

Input: path: "src/server.ts"

```
edits: [  
  { oldText: "port: 3000", newText: "port:  
config.port" },  
  { oldText: "// TODO: add auth", newText:  
"app.use(authMiddleware)" }  
]
```

Output: файл обновлён (только указанные фрагменты)

Отличие инструмента EditFile от WriteFile – он вносит минимальные изменения, не перезаписывая весь файл. Это снижает риск потери данных и делает изменения более предсказуемыми.

Терминальные инструменты

Bash – выполнение команд оболочки:

Input: command: "npm test"

Output: stdout + stderr + exit code

Это «швейцарский нож» Claude Code. Через Bash он может:

– Устанавливать зависимости

- Запускать тесты
- Компилировать код
- Управлять Git
- Работать с Docker
- Запускать линтеры и форматтеры
- Анализировать систему

Поисковые инструменты

Search (grep/ripgrep) – поиск по содержимому файлов:

```
Input:          query:          "function  
calculateDiscount"  
path: "src/"
```

Output: список файлов с совпадениями и
контекстом

Find (fd) – поиск файлов по имени:

```
Input: pattern: "*.test.ts"  
path: "src/"
```

Output: список путей к файлам

Веб-инструменты

WebSearch – поиск в интернете для получения актуальной информации. Этот инструмент помогает Claude Code находить документацию, решения проблем и актуальные версии библиотек.

ReadURL – чтение содержимого веб-страницы. Полезно для анализа API-документации, StackOverflow-ответов и README на GitHub.

МСП-инструменты

Model Context Protocol (МСП) позволяет подключать произвольные внешние инструменты. Подробнее о МСП – в Главе 13. Примеры:

- **Jira** – чтение и обновление задач
- **Slack** – отправка сообщений
- **GitHub** – управление пулл-реквестами
- **Database** – выполнение SQL-запросов
- **Kubernetes** – управление кластером

Координация инструментов

Ключевая способность Claude Code – не просто вызывать отдельные инструменты, а координировать их в последовательность для достижения комплексной цели. Рассмотрим пример:

Задача: «Исправь падающий тест test_user_creation»

Последовательность действий Claude Code:

```
1. Bash("npm test - test_user_creation")
```

Конец ознакомительного фрагмента.

Текст предоставлен ООО «Литрес».

Прочитайте эту книгу целиком, [купив полную легальную версию](#) на Литрес.

Безопасно оплатить книгу можно банковской картой Visa, MasterCard, Maestro, со счета мобильного телефона, с платежного терминала, в салоне МТС или Связной, через PayPal, WebMoney, Яндекс.Деньги, QIWI Кошелек, бонусными картами или другим удобным Вам способом.